



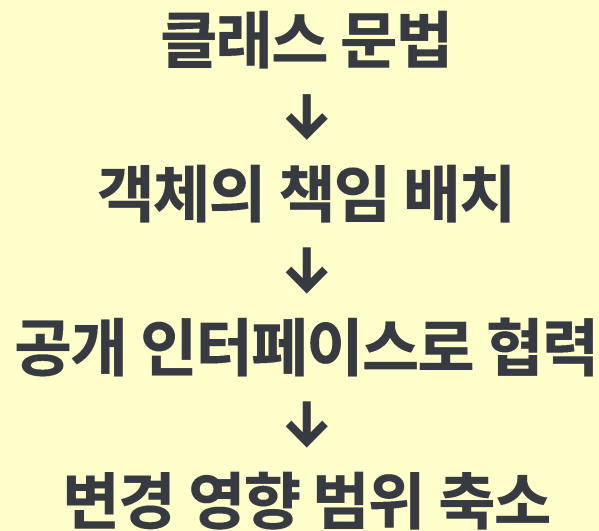
09. 객체지향 설계 원칙 (SOLID)

Heesung Oh

<https://github.com/3dapi>



- 책임과 협력
- 응집도와 결합도
- 인터페이스 설계
- 단일 책임 원칙(SRP)
- 개방-폐쇄 원칙(OCP)
- 리스코프 치환 원칙(LSP)
- 인터페이스 분리 원칙(ISP)
- 의존 역전 원칙(DIP)
- 객체지향 설계의 적용





- 객체지향 프로그램은 여러 객체가 협력하여 기능을 완성
- 책임: 객체가 알고 있거나 수행해야 할 일
- 협력: 여러 객체가 요청과 응답으로 기능을 완성하는 구조
- 메시지: 객체의 공개 기능 호출
- 객체는 자신의 상태와 규칙을 관리, 객체는 필요한 작업을 요청





- 책임은 객체가 알아야 하는 정보와 수행해야 하는 동작
- 상태와 상태를 다루는 규칙을 함께 검토
- 클래스 이름과 공개 인터페이스에서 책임이 드러나야 함

```
class HealthComponent
{
public:
    explicit HealthComponent(int maxHp);

    int TakeDamage(int damage);
    int Recover(int amount);
    int GetHp() const;
    int GetMaxHp() const;
    bool IsDead() const;

private:
    int hp;
    int maxHp;
};
```

- hp, maxHp → 체력 상태
- TakeDamage(), Recover() → 체력 변경 규칙
- IsDead() → 체력 상태 판단



- 역할은 협력 안에서 객체가 제공하는 능력
- 하나의 구체 클래스가 여러 역할을 수행할 수 있음
- 같은 역할을 여러 구체 클래스가 수행할 수 있음
- C++에서는 작은 추상 클래스로 역할을 표현 가능

```
class IDamageable
{
public:
    virtual ~IDamageable() = default;

    virtual int TakeDamage(int damage) = 0;
    virtual bool IsDead() const = 0;
};

void ApplyExplosionDamage(IDamageable& target, int damage)
{
    target.TakeDamage(damage);
}
```

- 호출 코드는 대상의 구체 타입보다 역할에 의존



9.1.3 데이터와 행위의 배치

```
// 외부가 체력 규칙을 처리  
int hp = target.GetHp();  
if (hp -= damage; hp < 0)  
{  
    hp = 0;  
}  
target.SetHp(hp);
```

```
// 상태를 가진 객체에 작업 요청  
const int actualDamage = target.TakeDamage(attackPower);
```

외부에서 처리	객체가 처리
체력 규칙을 외부가 알아야 함	체력 규칙을 내부에 유지
같은 보정 코드 반복 가능	변경 지점 집중
내부 표현과 결합	의미 있는 작업에 의존

- **핵심 규칙을 외부가 대신 수행하지 않도록 함**



책임 집중

Player

- 입력
- 이동
- 전투
- 체력
- 저장
- UI
- 사운드

책임 분리

Player

- HealthComponent
- AttackBehavior
- Inventory
- 외부 서비스와 협력

- 객체 수가 적다고 좋은 설계가 되는 것은 아님
- 클래스 수가 많다고 좋은 설계가 되는 것도 아님
- 함께 변경되는 상태와 행위는 모음
- 독립적으로 변경되는 책임은 분리
- 무엇을 나눌 것인가보다 왜 나누는가가 중요



기준	좋은 방향	나쁜 신호
응집도	하나의 목적 중심	관련 없는 기능 혼합
결합도	안정적인 역할에 의존	구체 구현과 내부 표현에 의존
변경	수정 지점이 좁음	한 변경이 여러 파일로 확산

높은 응집도

관련 상태 + 관련 행위

낮은 결합도

공개 인터페이스 + 역할 중심 의존

절대적인 수치가 아니라 설계를 판단하는 기준



- 하나의 목적을 중심으로 상태와 행위가 모이면 응집도가 높음
- 함수들이 같은 데이터와 같은 규칙을 다룸
- 클래스 이름과 멤버 목록이 같은 방향을 가져야 함

HealthComponent

- hp
- maxHp
- TakeDamage()
- Recover()
- IsDead()

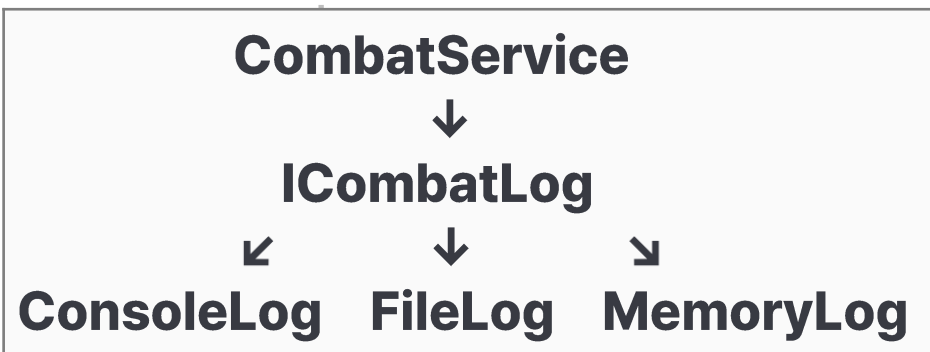
→ 모두 체력 상태와 규칙에 관련



```
class ICombatLog
{
public:
    virtual ~ICombatLog() = default;
    virtual void Write(std::string_view message) = 0;
};

class CombatService
{
public:
    explicit CombatService(ICombatLog& combatLog)
        : combatLog(combatLog)
    {
    }
}

private:
    ICombatLog& combatLog;
}
```



- 협력하려면 의존은 필요함
- 변하기 쉬운 구체 구현보다 안정적인 역할에 의존



```
int Player::CalculateDamage() const;  
int Monster::CalculateDamage() const;  
int BossMonster::CalculateDamage() const;
```

- 같은 계산 규칙이 여러 곳에 있으면
공식 변경 시 여러 클래스를 수정해야 함

```
class LevelDamagePolicy  
{  
public:  
    int Calculate(int attackPower, int level) const  
    {  
        return attackPower + level * 3;  
    }  
};
```

- 설계 품질은 요구 사항이 바뀔 때 드러남
- 변경 축을 찾아 변경 지점을 줄이는 것이 목표



- 인터페이스는 객체가 외부에 제공하는 사용 방법과 계약
- C++에서는 공개 멤버 함수, 매개변수, 반환 타입, const, 예외 사양, 소유권 타입이 모두 인터페이스에 포함
- 좋은 인터페이스는 객체가 무엇을 할 수 있는지 분명히 보여 줌
- 내부 구현을 변경해도 사용자 코드 영향이 적어야 함

인터페이스 요소	표현하는 의미
함수 이름	작업의 목적
매개변수	필요한 입력과 의존성
반환 타입	결과와 실패 표현
const	상태 변경 여부
참조·포인터 스마트 포인터	소유권과 수명

좋은 인터페이스

- 필요한 기능만 공개
- 실패 표현이 명확
- 소유권이 드러남
- 내부 표현에 의존하지 않음



- 객체 역할 수행에 필요한 기능만 공개
- 생성자와 공개 멤버 함수는 객체의 불변식(논리적 유효성) 유지

// 좋지 않은 공개

```
int& Hp();  
int& MaxHp();
```

```
health.Hp() = -500;  
health.MaxHp() = 0;
```

// 의미 있는 작업을 공개

```
int TakeDamage(int damage);  
int Recover(int amount);  
int GetHp() const;  
bool IsDead() const;
```

```
int TakeDamage(int damage)
```

```
{  
    if (damage <= 0)  
    {  
        return 0;  
    }  
}
```

// 불변식 유지

```
const int oldHp = hp;  
hp = std::clamp(hp - damage, 0, maxHp);  
return oldHp - hp;  
}
```



- 명령(command): 객체 상태를 변경하는 작업
- 질의(query): 상태를 관찰하여 값을 반환하는 작업
- 함수 이름만 보고 상태 변경 여부를 예측할 수 있어야 함

```
bool AddItem(int itemId);           // 명령
bool RemoveItem(int itemId);        // 명령
bool Contains(int itemId) const;    // 질의
std::size_t GetItemCount() const;  // 질의

int GetScore()
{
    return ++score; // 조회처럼 보이지만 상태 변경
}
```

- 조회 함수가 숨은 부작용을 만들지 않도록 설계



상황	표현 방법
값이 없을 수 있음	<code>std::optional</code>
여러 실패 이유 구분	결과 열거형 또는 결과 객체
유효한 객체 생성 불가	예외
호출자가 지켜야 하는 조건	전제 조건 문서화와 검증

- `std::optional`: 값이 존재할 수도, 없을 수도 있는 상태를 표현
- 탐색 실패 시 `nullptr`나 임의의 매직 넘버(예: -1)를 반환하던 관행을 대체

```
std::optional<Item> FindById(int id) const
{
    for (const Item& item : items)
    {
        if (item.id == id) {
            return item;
        }
    }
    return std::nullopt;
}
```

```
if (auto item = inventory.FindById(1001))
{
    std::cout << item->name << '\n';
}
```



- C++ 인터페이스는 타입으로 소유권과 수명 관계를 표현
- 소유권을 숨기지 않으면 객체 파괴 책임이 분명해짐

선언	의미
const T&	읽기 전용, 소유권 없음
T*	null 가능, 일반적으로 비소유
std::unique_ptr<T> 반환	고유 소유권 전달
std::unique_ptr<T> 매개변수	소유권 이전
std::shared_ptr<T>	공유 소유권

```
void Render(const GameObject& object);  
void SetTarget(GameObject* target);  
std::unique_ptr<GameObject> CreateObject();  
void AddObject(std::unique_ptr<GameObject> object);  
std::shared_ptr<Resource> LoadSharedResource();
```




● Single Responsibility Principle

- ◆ 클래스가 하나의 변경 이유를 가지도록 설계
- ◆ 함수 하나만 가져야 한다는 뜻이 아님
- ◆ 서로 다른 이유로 변경되는 기능이 한 클래스에 모이면 수정 범위가 넓어짐

```
class Player
{
public:
    void Attack();
    void TakeDamage(int damage);

    void DrawStatusPanel();
    void SaveAsJson();
    void SendToServer();
    void PlayAttackSound();
};
```

Player

- ├ 전투 규칙
- ├ UI 출력
- ├ 저장 형식
- ├ 네트워크
- └ 사운드

→ 서로 다른 변경 이유가 한 클래스에 집중



- 클래스가 누구의 요구에 의해 바뀌는지 확인
- 전투, UI, 저장, 사운드, 네트워크는 서로 다른 변경 축
- 변경 이유별로 책임을 분리하면 충돌과 회귀 오류 감소

```
class Player
{
public:
    void Attack(IDamageable& target);
    int TakeDamage(int damage);
    int GetHp() const;
};

class PlayerStatusView
{
public:
    void Draw(const Player& player) const;
};

class IPlayerRepository
{
public:
    virtual ~IPlayerRepository() = default;
    virtual void Save(const Player& player) = 0;
};
```

Player	→ 게임 상태와 전투 규칙
PlayerStatusView	→ 표시
Repository	→ 저장 방식
AudioService	→ 사운드 출력



- SRP를 지나치게 적용하면 불필요한 클래스가 늘어남
- 변경 빈도, 재사용, 시험 요구에 따라 분리 범위를 결정
- 피해, 회복, 사망 판정은 모두 체력 규칙이라는 하나의 책임으로 볼 수 있음

분리할 것

→ 독립적으로 변경되는 책임

묶을 것

→ 항상 함께 변경되는 상태와 규칙

- 가장 작은 클래스를 만드는 원칙이 아님



● Open-Closed Principle

- ◆ 확장에는 열려 있고, 기존 코드 변경에는 닫혀 있는 구조 지향
- ◆ 반복해서 변하는 지점을 안정적인 인터페이스와 교체 가능한 구현으로 분리
- ◆ 기존 코드를 절대로 수정하지 말라는 뜻은 아님

```
class Player
{
public:
    int CalculateDamage() const
    {
        switch (attackType)
        {
            case AttackType::Melee:
                return strength * 2;

            case AttackType::Ranged:
                return dexterity + arrowPower;
        }
        return 0;
    }
}
```

새 공격 방식 추가
↓
기존 switch 수정
↓
기존 코드 변경
↓
새 구현 클래스 추가
↓
안정된 코드 영향 감소



```
switch (attackType)
{
case AttackType::Melee:
    // 근접 공격
    break;

case AttackType::Ranged:
    // 원거리 공격
    break;
}
```

- 조건문 자체가 문제는 아님
- 같은 종류의 분기가 여러 곳에 반복되는 것이 문제
- 새 종류 추가 시 여러 분기를 동시에 수정해야 함

공격 방식 지식
├─ 피해 계산
├─ 사운드
├─ 아이콘
└─ 설명 문구





- 여러 구현이 필요하거나 변화 가능성이 높은 지점에 공통 계약 정의
- 모든 클래스에 인터페이스를 만드는 것이 추상화의 목적은 아님

```
class IAttackBehavior
{
public:
    virtual ~IAttackBehavior() = default;

    virtual int CalculateDamage() const = 0;
};

class MeleeAttack : public IAttackBehavior
{
public:
    int CalculateDamage() const override;
};
```

- 새로운 공격은 새 구현 클래스로 추가



- Player는 공격 행동을 고유 소유
- 실행 중 다른 공격 행동으로 교체 가능
- 공격 종류 추가가 Player의 분기 증가로 이어지지 않음

```
class Player
{
public:
    explicit Player(std::unique_ptr<IAttackBehavior> attackBehavior)
        : attackBehavior(std::move(attackBehavior))
    {
    }

    int CalculateDamage() const
    {
        return attackBehavior->CalculateDamage();
    }

private:
    std::unique_ptr<IAttackBehavior> attackBehavior;
};
```





- 타입의 종류를 표현할 때는 상속 검토
- 교체 가능한 기능과 정책은 구성 우선 검토

상속만 사용

Player

```
├─ MeleePlayer
├─ RangedPlayer
├─ MagicPlayer
└─ PoisonPlayer
```

공격 + 방어 조합 → 파생 타입 수 폭발적 증가

구성 사용

Player

```
├─ IAttackBehavior
└─ IDefenseBehavior
```





- 기반 타입을 사용하는 코드가 파생 타입으로 교체되어도 올바르게 동작해야 함
- 공개 상속은 기반 타입의 의미와 계약을 이어받는 관계
- 함수 형식이 맞아도 동작 의미가 깨지면 치환 가능하지 않음

```
class IMovable
{
public:
    virtual ~IMovable() = default;

    virtual void MoveTo(float x, float y) = 0;
    virtual float GetX() const = 0;
    virtual float GetY() const = 0;
};
```

```
void MoveToOrigin(IMovable& object)
{
    object.MoveTo(0.0f, 0.0f);

    if (object.GetX() != 0.0f || object.GetY() != 0.0f)
    {
        throw std::runtime_error("object did not move");
    }
}
```

- 모든 IMovable 구현은 이 기대를 만족해야 함



9.6.2 계약(객체간 약속) 유지

계약 요소	의미
수행 작업	함수가 해야 하는 일
사전 조건	호출 전에 만족해야 할 조건
사후 조건	호출 후 보장되는 결과
불변식	객체가 항상 유지해야 하는 규칙
실패 표현	실패를 전달하는 방식

```
class Wall : public IMovable
{
public:
    void MoveTo(float, float) override
    {
        // 이동하지 않음
    }
};
```

- Wall이 이동할 수 없다면 IMovable 역할 자체가 적절하지 않음





- 파생 타입은 기반 타입보다 더 강한 사전 조건을 요구하면 안 됨
- 파생 타입은 기반 타입이 보장하는 사후 조건을 약화하면 안 됨

```
class IDamageable
{
public:
    virtual ~IDamageable() = default;

    // damage >= 0이면 호출 가능
    virtual int TakeDamage(int damage) = 0;
};
```

```
class FragileObject : public IDamageable
{
public:
    int TakeDamage(int damage) override
    {
        if (damage > 100)
        {
            throw std::invalid_argument("damage must not exceed 100");
        }
        return damage;
    }
};
```

// 파생 타입에서 $damage \leq 100$ 만 허용
// 기반 타입보다 강한 사례



9.6.4 잘못된 상속 관계

- 코드 재사용만 보고 상속하지 않음
- 공통 상태와 실제 역할을 구분

class MovableObject

```
{  
public:  
    virtual ~MovableObject() = default;  
    virtual void MoveTo(float x, float y) = 0;  
};
```

class StaticDecoration : public MovableObject

```
{  
public:  
    void MoveTo(float, float) override  
    {  
        throw std::logic_error("static decoration cannot move");  
    }  
}
```

class IMovable

```
{  
public:  
    virtual ~IMovable() = default;  
    virtual void MoveTo(float x, float y) = 0;  
};
```

class Player : public GameObject , public IMovable

```
{  
public:  
    void MoveTo(float x, float y) override  
    {  
        SetPosition(x, y);  
    }  
}
```

개선 구조

GameObject → 공통 상태
IMovable → 이동 역할
Player → GameObject + IMovable
StaticDecoration → GameObject

class StaticDecoration : public GameObject

```
{  
};
```



● Interface Segregation Principle

- ◆ 사용자가 필요하지 않은 기능에 의존하도록 강요하지 않음
- ◆ 하나의 거대한 인터페이스보다 역할별 인터페이스를 검토

```
class IGameEntity
{
public:
    virtual void Update() = 0;
    virtual void Render() const = 0;
    virtual void ProcessInput() = 0;
    virtual int TakeDamage(int damage) = 0;
    virtual void Save() const = 0;
};
```

```
class Wall : public IGameEntity
{
    // ...
    void ProcessInput() override
    {
        // 사용하지 않음
    }
    void Save() const override
    {
        // 사용하지 않음
    }
};
```

- Wall, ParticleEffect 등에는 필요 없는 기능이 생길 수 있음



- 각 시스템은 자신이 실제 사용하는 역할에만 의존하도록 구성

```
class IUpdatable
```

```
{  
public:  
    virtual ~IUpdatable() = default;  
    virtual void Update() = 0;  
};
```

```
class IRenderable
```

```
{  
public:  
    virtual ~IRenderable() = default;  
    virtual void Render() const = 0;  
};
```

```
class IDamageable
```

```
{  
public:  
    virtual ~IDamageable() = default;  
    virtual int TakeDamage(int damage) = 0;  
};
```

```
void UpdateObject(IUpdatable& object);  
void RenderObject(const IRenderable& object);  
void ApplyDamage(IDamageable& object, int damage);
```



- 작은 역할 인터페이스는 필요한 능력을 조합하기 쉬움
- 지나치게 잘게 나누면 호출 관계와 타입 수가 증가할 수 있음
- 분리 기준은 구현 클래스의 편의보다 사용하는 코드가 필요한 기능

```
class Player : public IUpdatable,  
               public IRenderable,  
               public IInputReceiver,  
               public IDamageable  
{  
    // ...  
};
```





● Dependency Inversion Principle

- ◆ 상위 수준의 정책 코드가 하위 수준의 구체 구현에 직접 의존하지 않도록 함
- ◆ 상위 수준과 하위 수준이 추상화에 의존

나쁜 방향

```
CombatService —> std::mt19937  
CombatService —> std::cout
```

개선 방향

```
CombatService —> IRandomSource <— MtRandomSource  
CombatService —> ICombatLog <— ConsoleCombatLog
```




9.8.1 구체 구현에서 추상화로

CombatService가 직접 아는 것

- 난수 엔진의 구체 타입
- 콘솔 출력

문제

- 실행 환경 변경 시 CombatService 수정
- 시험에서 난수 결과 제어

```
class IRandomSource
```

```
{  
public:  
    virtual ~IRandomSource() = default;  
    virtual int NextInt(int min, int max) = 0;  
};
```

```
class ICombatLog
```

```
{  
public:  
    virtual ~ICombatLog() = default;  
    virtual void Write(std::string_view  
message) = 0;  
};
```

```
class CombatService
```

```
{  
public:  
    CombatService(IRandomSource& randomSource,  
        ICombatLog& combatLog)  
        : randomSource(randomSource),  
          combatLog(combatLog)  
    {  
    }  
  
    void Attack(Character& attacker, Character& target);  
  
private:  
    IRandomSource& randomSource;  
    ICombatLog& combatLog;  
};
```



- 필요한 객체를 내부에서 직접 생성하지 않고 외부에서 전달받음
- 생성자 주입은 반드시 필요한 의존성을 표현하기 좋음

```
class CombatService
{
public:
    CombatService(IRandomSource& randomSource,
                  ICombatLog& combatLog)
        : randomSource(randomSource),
          combatLog(combatLog)
    {
    }
}
```

```
private:
    IRandomSource& randomSource;
    ICombatLog& combatLog;
};
```

```
int main()
{
    MtRandomSource randomSource;
    ConsoleCombatLog combatLog;
    CombatService combatService(randomSource,
                                combatLog);
}
```

```
main()
├─ 난수 구현 생성
├─ 기록 구현 생성
└─ CombatService에 주입
```



- 설계 원칙은 실제 코드에서 함께 작용
- 책임 분리 → 응집도 상승
- 작은 인터페이스 → 결합도와 변경 범위 감소
- 의존성 주입 → 실행 환경과 시험 환경 분리
- 목표는 클래스 수를 늘리는 것이 아니라 변경 축을 구분하는 것

초기 구조

Player

```
├─ 공격 종류 판정
├─ 전역 난수 사용
├─ 대상 체력 직접 변경
└─ 콘솔 출력
```

개선 구조

Character

```
├─ HealthComponent
├─ IAttackBehavior
├─ CombatService
│   └─ IRandomSource
│       └─ ICombatLog
```



- 각 객체가 하나의 변경 이유를 가지도록 분리

HealthComponent

- 현재 체력과 최대 체력 유지
- 피해 적용과 사망 판정

IAttackBehavior

- 공격 피해량 계산
- 공격 방식별 확률 규칙

Character

- 이름과 체력 소유
- 공격 행동 객체 소유
- 공격 행동 교체

CombatService

- 공격자와 대상의 협력 순서 조정
- 전투 결과 기록 요청

IRandomSource

- 범위 안의 난수 제공

ILogCombat

- 전투 결과 출력 또는 저장





- 공격 방식은 상속 계층보다 구성으로 교체
- 전역 난수와 전역 기록 객체를 제거
- 숨은 의존성을 생성자 매개변수로 드러냄

Character

- └ HealthComponent
- └ IAttackBehavior

CombatService

- └ IRandomSource&
- └ ICombat



- 모든 구체 클래스에 인터페이스를 만들 필요는 없음
- 실제로 교체할 필요가 있는 부분만 분리
- 요구 사항이 확정되지 않았는데 잘못된 확장 지점을 먼저 고정하지 않음

분리 가치가 높은 지점	이유
공격 계산	여러 공격 방식 존재
난수 공급	시험에서 제어 필요
전투 기록	출력 대상 교체 가능

구체 클래스로 둘 수 있는 지점	이유
HealthComponent	안정적인 체력 규칙
AttackRoll	단순 결과 값
PlayerData	단순 데이터 전달

좋은 추상화는 실제 변경과 중복이 드러난 지점에 도입

- 추상화도 유지 비용을 가짐



- 객체지향 설계: 기능을 책임으로 분배, 객체가 인터페이스로 협력
 - 객체는 자신의 상태와 규칙을 관리
 - 역할은 협력 안에서 객체가 제공하는 능력
 - 작은 인터페이스는 서로 다른 구체 객체를 같은 방식으로 사용
 - 응집도 높은 클래스는 하나의 목적과 관련된 상태와 행위를 모음
 - 결합도 낮은 객체는 다른 객체의 내부 표현과 구체 구현을 적게 얹
 - 인터페이스는 정상 동작, 실패 방식, 소유권과 수명을 사용자에게 제공
-
- SRP: 하나의 변경 이유를 가지도록 책임을 분리
 - OCP: 반복해서 변하는 지점을 추상, 새 구현으로 확장
 - LSP: 파생 타입이 기반 타입의 계약을 유지하도록 요구



- ISP: 필요 없는 함수에 의존하지 않도록 역할별 인터페이스 구성
- DIP: 핵심 정책이 세부 기술에 직접 의존하지 않음
- 의존성 주입은 실행 환경과 시험 환경에서 구현을 교체할 수 있게
- 상속은 is-a 관계와 동적 대체가 필요할 때 사용
- 구성은 교체 가능한 기능과 정책을 표현할 때 유용
- 설계 원칙은 변경 가능성과 복잡도로 판단
- 작은 프로그램에 불필요한 계층을 먼저 만들 필요는 없음
- 함께 바뀌는 부분과 독립적으로 바뀌는 부분을 관찰
- 변경 비용과 구현 복잡도의 균형을 기준으로 적용

